Aging Load: Iscandar vs Buddy

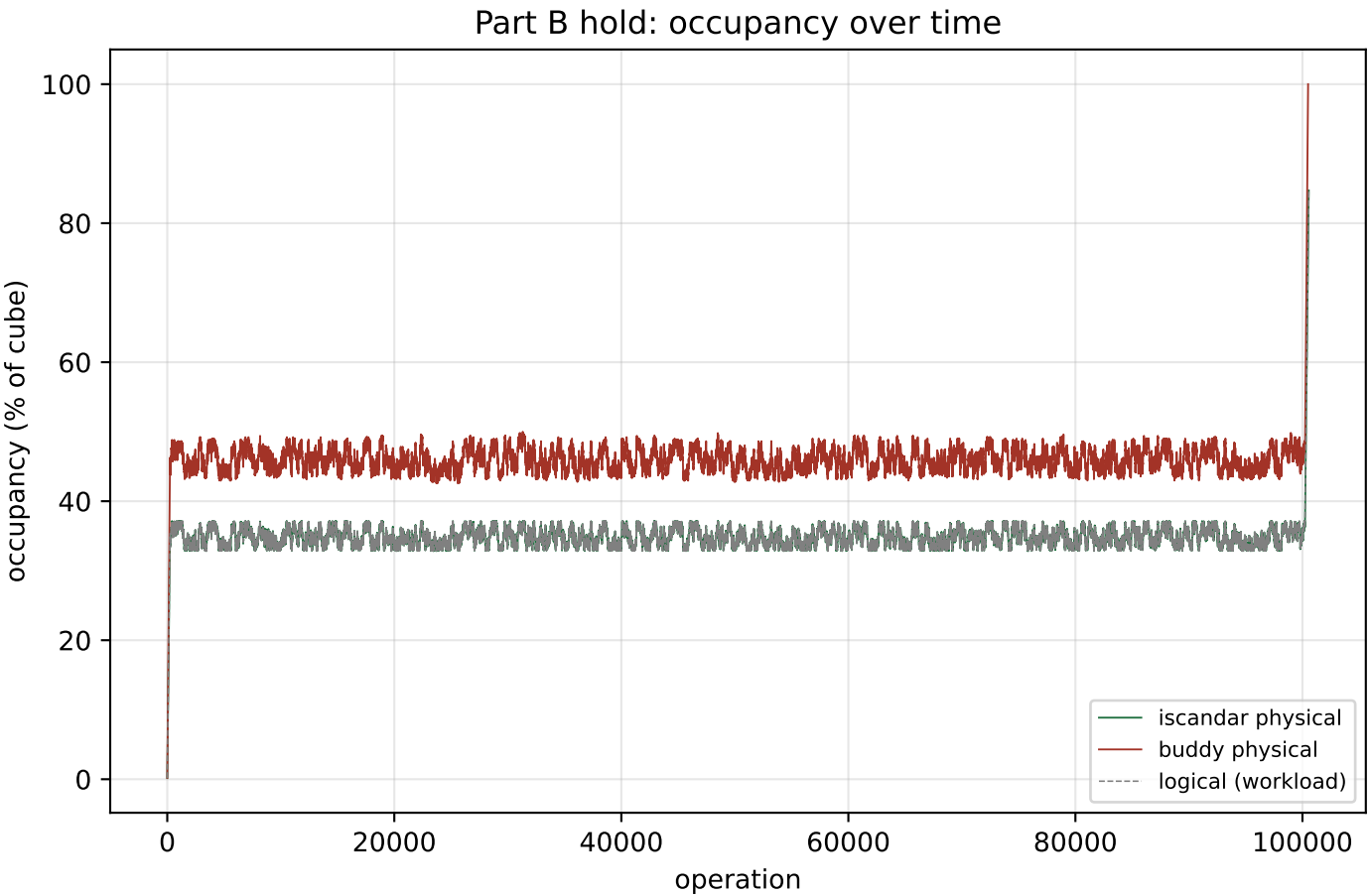
```
Load: Aging Load (author's shelf, 2026-10-26):
  the control-block profile (33..64 sectors)
  held at 35% logical for 100,000 operations -
  the endurance case: does fragmentation grow
  over a system's lifetime? (Its sprint is the
  control-block sprint by construction: same
  seed, same shape.)

Engines: the full Iscandar engine (all modes,
  exact fit) vs the classic binary buddy.
Arena: 32,768 sectors (128 KiB at 4 B/sector).
The same seeded stream drives both engines.

Part A - the sprint (fill to 85% logical, churn,
push; first failed allocation loses):
  buddy    stalled op 512 (ramp) at 100.00% physical
  iscandar stalled op 892 (churn) at 87.22% physical
  margin 380 ops to iscandar.

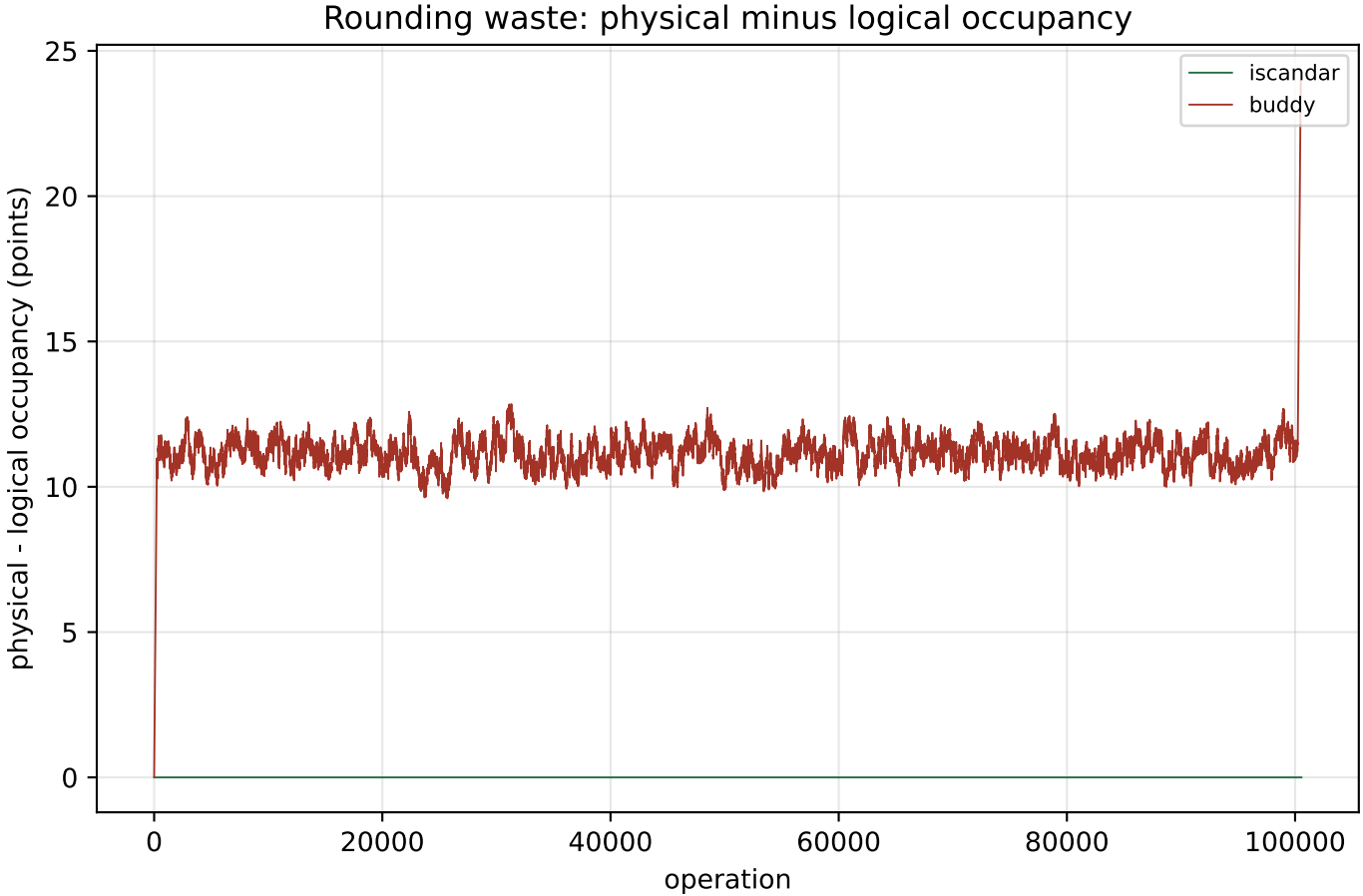
Part B - the steady-state hold (fill to 35%
logical, 100000 ops in a +/-2-point band, push):
  refusals during the hold: iscandar 0, buddy 0
  push stall: buddy at 100.00% physical, largest
            free block 0 sectors
  push stall: iscandar at 84.72% physical, largest
            free run 49 sectors

GammaCheck green in both parts (block checks
every op, full-ladder checks at boundaries).
```


Part B hold: occupancy over time
occupancy (% of cube)
operation
0
20
40
60
80
100
0
20000
40000
60000
80000
100000
iscandar physical
buddy physical
logical (workload)

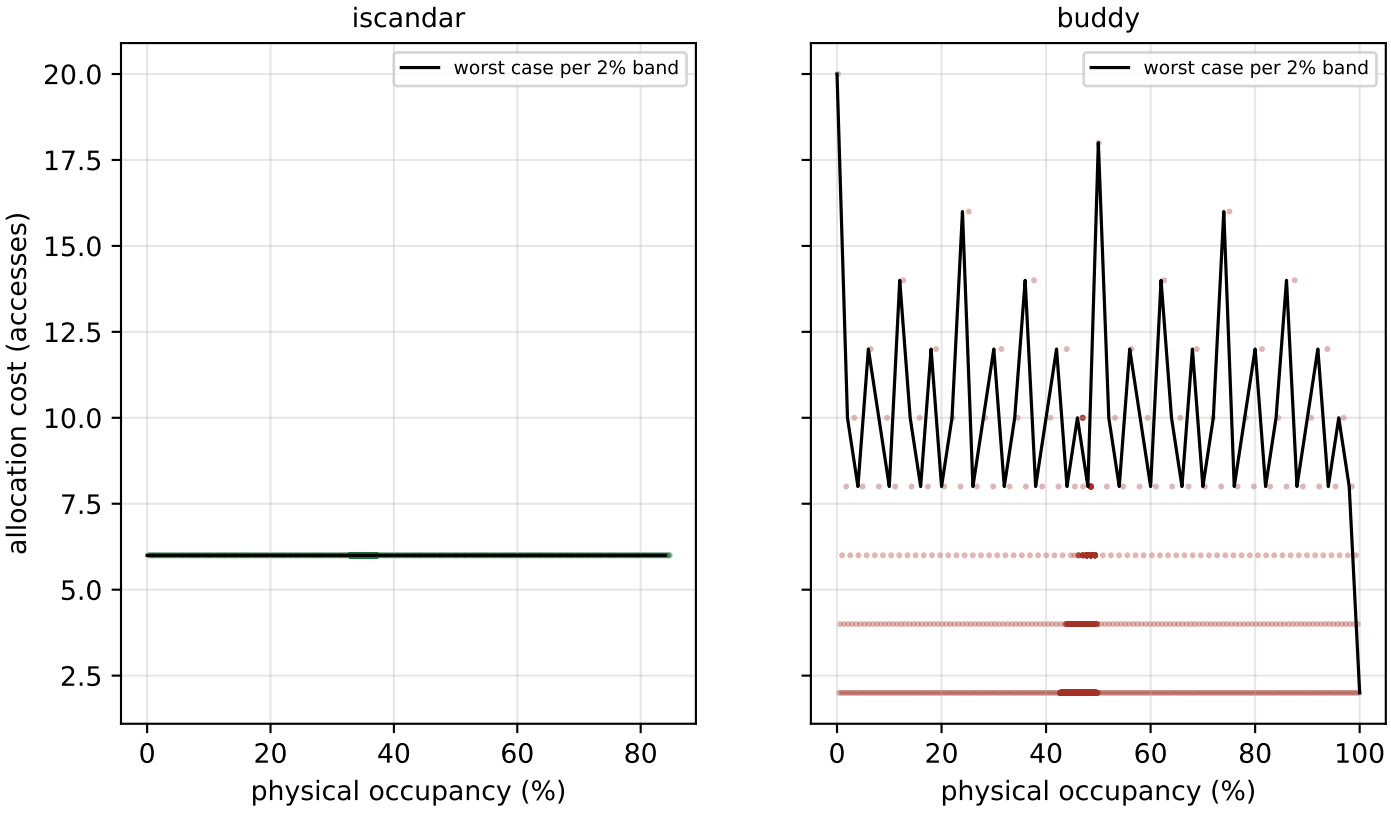
Commentary - occupancy over time

The gray dashed line is the workload itself, held at 35% logical. Iscandar's green line lies on top of it (mean 34.8% physical), because exact fit makes physical occupancy equal to logical occupancy. Buddy's red line rides above it: mean 45.9% physical, ranging 42.6 to 50.0%. The vertical distance between the red line and the workload is space Buddy has consumed without holding data.


Rounding waste: physical minus logical occupancy
physical - logical occupancy (points)
operation
iscandar
buddy
0
5
10
15
20
25
0
20000
40000
60000
80000
100000

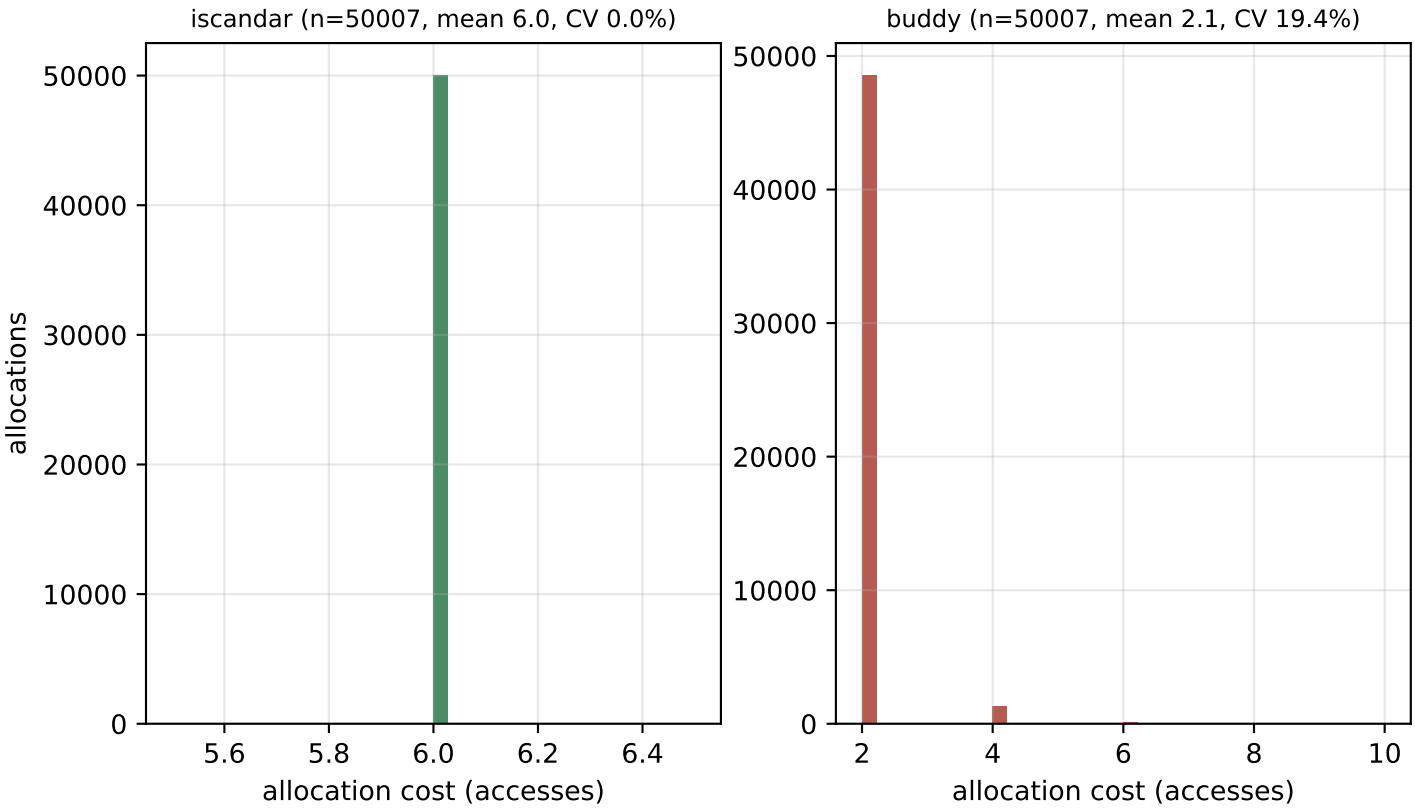
Commentary - rounding waste

Iscandar is a flat line on zero for the entire run. Buddy averages 11.1 points of pure waste and peaks at 12.8 - space that is neither data nor usable, only rounding.


Allocation cost against occupancy (hold + push; units differ per engine, see closing page)
iscandar
buddy
worst case per 2% band
worst case per 2% band
allocation cost (accesses)
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0
20
40
60
80
0
20
40
60
80
100
physical occupancy (%)
physical occupancy (%)

Commentary - cost against occupancy

Two panels with a black worst-case envelope. Iscandar's envelope stays between 6 and 6 touches across every occupancy band it visits; the price does not blow up as the cube fills. Buddy's envelope stays between 2 and 20 in its own tree-visit units (the levels are not comparable across panels; the shapes are). The readable point is the absence of a cliff: neither engine's worst case runs away with occupancy under this load.


Allocation cost distribution in the hold (steady state)
iscandar (n=50007, mean 6.0, CV 0.0%)
buddy (n=50007, mean 2.1, CV 19.4%)
allocations
0
10000
20000
30000
40000
50000
5.6
5.8
6.0
6.2
6.4
2
4
6
8
10
allocation cost (accesses)
allocation cost (accesses)

Commentary - cost distributions

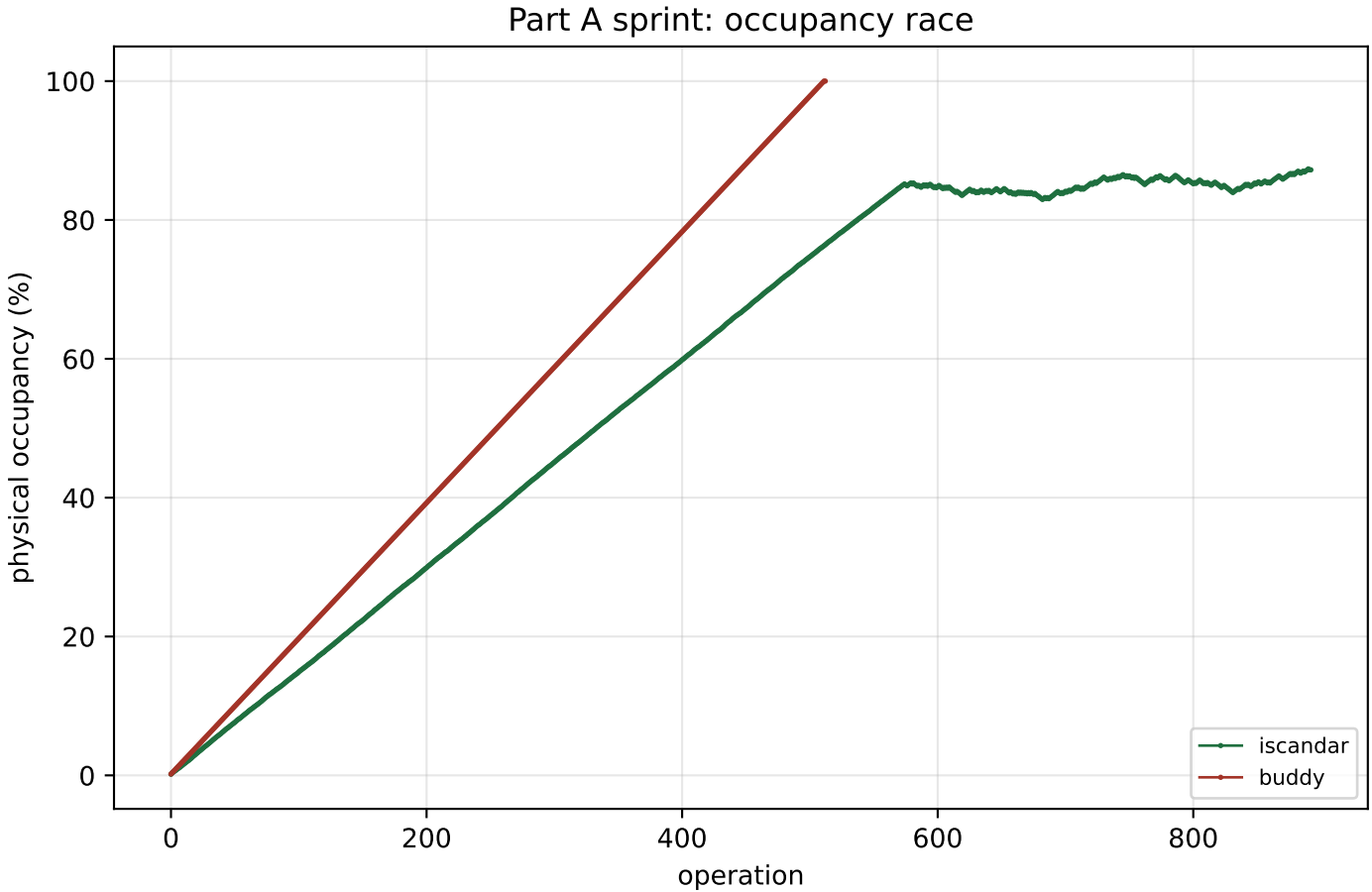
In the hold, Iscandar's allocation cost has median 6, with the middle half between 6 and 6 and the 90th percentile at 6. Buddy's has median 2, middle half 2 to 2, 90th percentile 2. The stability sigma on the closing page states the same shapes as a single number per series.


Largest free run over time (hold + push): fragmentation made visible
iscandar
buddy
largest free run (sectors)
30000
25000
20000
15000
10000
5000
0
0
20000
40000
60000
80000
100000
operation

Commentary - largest free run

The fragmentation picture. Through the hold, Iscandar's largest contiguous free run has median 781 sectors; Buddy's largest free block has median 16384. At the push stalls, Buddy dies with a largest free block of 0 sectors at 100.00% physical, Iscandar with a largest free run of 49 at 84.72% physical.

Graph 5 is the one that makes the RTOS applications engineer stop and look. The occupancy graphs show what Buddy wastes; the largest-free-run graph shows what that waste costs - the size of the biggest promise each allocator can still keep.


Part A sprint: occupancy race
physical occupancy (%)
operation
0
20
40
60
80
100
0
200
400
600
800
iscandar
buddy

Commentary - the sprint

buddy stalls first, at op 512 (100.00% physical); iscandar continues alone and stalls at op 892 (87.22% physical). The margin is 380 ops to iscandar.

Numbers and caveats

```
Numbers (population statistics over successful ops)

sprint iscandar alloc: n=741 mean 6.00 var 0.00 CV 0.0% stability inf sigma, range 6..6

sprint buddy alloc: n=512 mean 4.00 var 7.85 CV 70.1% stability 1.4 sigma, range 2..20

hold   iscandar alloc: n=50007 mean 6.00 var 0.00 CV 0.0% stability inf sigma, range 6..6
hold   iscandar free:  n=49993 mean 4.00 var 0.00 CV 0.0% stability inf sigma, range 4..4

hold   buddy alloc: n=50007 mean 2.07 var 0.16 CV 19.4% stability 5.2 sigma, range 2..10
hold   buddy free:  n=49993 mean 2.03 var 0.04 CV 9.8% stability 10.2 sigma, range 2..6

Counting and caveats

- Iscandar accesses are sector-word touches on
  the engine path: probe words in the search plus
  the commit's word touches, per the unified
  model in gaussian_race.py's header. Gate
  (counter) reads are a separate series and are
  not blended into accesses.
- Buddy accesses are its tree-work analog units
  (buddy.py), NOT the same physical units. Read
  across engines by shape, CV, and outcome;
  read levels and sigmas only within one engine
  against its own bound (the two-series
  discipline adopted 2026-10-26).
- Stability sigma is mean / SD (the reciprocal
  of CV): how many of its own standard
  deviations the mean cost sits above zero. It
  is a stability ratio, not a defect-rate
  sigma; a defect sigma needs a declared limit.
- Placement completeness (post-amendment): an
  Iscandar failure means no contiguous free run
  of the request existed, or the request
  exceeded all free space. It never fails facing
  an exact-fit hole.
- One seeded stream drove both engines in each
  part (seed 1). GammaCheck green throughout.
```